6주차

SELECT ④
GROUP BY — 묶어서 요약하기

- GROUP BY — 그룹별 집계의 원리
- 진료과별 · 유형별 · 월별로 세기
- HAVING — 그룹에 거는 조건
- Table 1 맛보기 — 논문 첫 표의 정체

> **이번 주 대표 질문**
> 진료과별 환자 수는 각각 몇 명인가?

GROUP BY의 원리 — 나눠 담고, 각각 요약

원리

GROUP BY가 하는 일

① 지정한 컬럼의 값이 같은 행끼리 **바구니에 나눠 담고**

② 바구니마다 집계함수를 **한 번씩** 적용

③ 바구니 수만큼의 행을 반환

철칙 하나

SELECT에는 ① **GROUP BY에 쓴 컬럼**과 ② **집계함수**만 올 수 있습니다.

그룹당 한 행으로 접히는데, 그룹 안에서 값이 제각각인 일반 컬럼은 '어느 값을 보여줄지' 정할 수 없기 때문.

GROUP BY 첫 실전

대표 질문

임상 질문

진료과별 내원 건수와 환자 수는?

```sql
SELECT department,
       COUNT(*) AS 내원건수,
       COUNT(DISTINCT patient_id) AS 환자수
FROM visit
GROUP BY department
ORDER BY 내원건수 DESC;
```

department	내원건수	환자수
내과	57	23
신경과	22	10
가정의학과	10	6
응급의학과	7	7
외과	2	2
정형외과	1	1
소아청소년과	1	1

총 7행

내원건수 ≠ 환자수에 주목 — 만성질환 외래는 같은 환자가 반복 방문합니다. `COUNT(DISTINCT patient_id)`가 '사람 수'입니다. 행 단위(내원)와 사람 단위를 혼동하면 유병률이 뻥튀기됩니다.

그룹별 다중 집계

임상 질문

내원 유형(외래/입원/응급)별로 평균 재원일수까지

```
SELECT visit_type,
       COUNT(*) AS 건수,
       ROUND(AVG(julianday(discharge_date)
         - julianday(visit_date)), 1)
         AS 평균재원일
FROM visit
GROUP BY visit_type;
```

visit_type	건수	평균재원일
외래	71	∅
응급	8	∅
입원	21	11.7

총 3행

외래·응급의 평균재원일은 NULL — discharge_date가 없으니 AVG가 계산할 대상이 없습니다. 지난주의 'NULL은 조용히 빠진다'가 그룹별로도 그대로 적용.

날짜 함수 × GROUP BY

임상 질문

월별 내원 추이 — 시간 축으로 묶기

```sql
SELECT strftime('%Y-%m', visit_date) AS 월,
       COUNT(*) AS 내원건수
FROM visit
WHERE visit_date >= '2025-01-01'
GROUP BY strftime('%Y-%m', visit_date)
ORDER BY 월;
```

월	내원건수
2025-01	3
2025-02	1
2025-03	3
2025-04	2
2025-05	7
2025-06	4
2025-07	3

… 총 12행 중 7행 표시

`strftime('%Y-%m', …)`으로 '연-월'만 뽑아 묶으면 월별 통계. 감염병 유행 곡선, 계절성 분석이 전부 이 문형입니다.

HAVING — 그룹에 거는 조건

대표 질문

임상 질문

10건 이상 처방된 약물만 보여줘

```
SELECT drug_name,
       COUNT(*) AS 처방건수
FROM prescription
GROUP BY drug_name
HAVING COUNT(*) >= 10
ORDER BY 처방건수 DESC;
```

drug_name	처방건수
암로디핀	46
아토르바스타틴	25
메트포르민	24
클로피도그렐	20
아세트아미노펜	17

총 5행

WHERE는 **행**을 거르고(집계 전), HAVING은 **그룹**을 거릅니다(집계 후). '처방건수 10건 이상'은 집계가 끝나야 알 수 있으니 HAVING 자리입니다.

WHERE vs HAVING — 언제 어느 쪽?

WHERE (집계 전, 행 필터)

- "2024년 내원 중에서" — 원본 행 조건
- 집계함수를 **쓸 수 없다**
- 먼저 실행되어 계산량을 줄인다

HAVING (집계 후, 그룹 필터)

- "환자 수가 5명 이상인 진료과만" — 요약값 조건
- 집계함수를 **쓰기 위한** 자리
- GROUP BY 다음에 실행된다

실행 순서: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. "2024년(WHERE) 내원 기준으로, 내원이 10건 이상인(HAVING) 진료과"처럼 둘을 함께 쓰는 게 보통입니다.

다중 컬럼 GROUP BY — Table 1 맛보기

Table 1

임상 질문

성별 × 연령대 분포 — Table 1의 첫 줄 만들기

```
SELECT sex,
  CASE WHEN 2026 - CAST(substr(birth_date,1,4) AS
INT) >= 65
       THEN '65세 이상' ELSE '65세 미만'
  END AS 연령군,
  COUNT(*) AS 명수
FROM patient
GROUP BY sex, 연령군
ORDER BY sex, 연령군;
```

sex	연령군	명수
F	65세 미만	14
F	65세 이상	14
M	65세 미만	16
M	65세 이상	6

총 4행

논문의 Table 1(연구대상자 특성표)은 결국 **CASE로 범주를 만들고 GROUP BY로 세는 것**의 반복입니다. 13주차에 코호트 전용 Table 1을 완성합니다.

지난 3주의 총결산 — 필터 × CASE × 집계 × 그룹

임상 질문

진료과별로 '고혈압(sbp≥140) 측정 비율'까지 — 종합 연습

```sql
SELECT v.department,
       COUNT(*) AS 측정수,
       ROUND(100.0 * SUM(CASE WHEN t.sbp >= 140
         THEN 1 ELSE 0 END) / COUNT(*), 1)
         AS 고혈압비율
FROM vital t, visit v
WHERE t.visit_id = v.visit_id
GROUP BY v.department
ORDER BY 고혈압비율 DESC;
```

department	측정수	고혈압비율
신경과	46	100
내과	69	21.7
가정의학과	10	10
정형외과	5	0
응급의학과	14	0
외과	4	0
소아청소년과	1	0

총 7행

`FROM vital t, visit v` + WHERE 연결은 **암시적 조인** — 11주차에 배울 `JOIN … ON`과 동작이 같습니다. WHERE·CASE·SUM·GROUP BY가 한 쿼리에 모두 들어간 형태로, 중간고사 최고 난도 수준입니다.

연습문제

실습

- **Q1.** KCD 코드별 진단 건수를 많은 순으로
- **Q2.** 검사 종류별 평균 결과값과 검사 건수는?
- **Q3.** 5회 이상 내원한 환자의 ID와 내원 횟수는?

  힌트: GROUP BY patient_id + HAVING
- **Q4.** 혈액형별 환자 수 분포는?
- **Q5.** 연도별 입원 건수 추이는?

  힌트: strftime('%Y', …)

중간고사 안내: 9주차 시험 범위는 1~8주차 전체. SELECT 4부작(3~6주차)이 배점의 중심입니다.

이번 주 용어 정리

용어

용어	뜻
GROUP BY	지정 컬럼의 값이 같은 행끼리 묶어 그룹당 한 행으로 요약
HAVING	집계가 끝난 그룹에 거는 조건 (WHERE는 집계 전 행에)
COUNT(DISTINCT x)	중복을 빼고 센 개수 — '내원 수'가 아닌 '사람 수'를 셀 때
strftime()	날짜에서 원하는 부분을 추출하는 SQLite 함수 — '%Y-%m'이면 연-월
실행 순서	FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
암시적 조인	FROM a, b + WHERE 연결 — JOIN…ON(11주차)과 같은 동작의 옛 문법
Table 1	연구대상자 기저 특성표 — CASE로 범주를 만들고 GROUP BY로 세면 완성

6주차 정리

- GROUP BY = 같은 값끼리 담아 **그룹당 한 행**으로 요약
- **내원 수 ≠ 환자 수** — '사람 수'는 COUNT(DISTINCT patient_id)로 센다
- WHERE는 행(집계 전), HAVING은 그룹(집계 후) — 실행 순서가 근거
- CASE로 범주 만들고 GROUP BY로 세면 그것이 **Table 1**

과제 Colab 6주차 Q1~Q5

다음 주 CREATE TABLE — 이번엔 우리가 테이블을 만든다: 연구 데이터셋 설계